FINOPTIX END EVALUATION

Comprehensive Project Report

Finance and Data Analytics Club

Indian Institute of Technology Kanpur

MENTORS

Keshav Bansal • Kushagra Tiwari
Aarush Singh • Ankit Kumar

About

This project presents an integrated framework for quantitative finance, designed to systematize the investment process from stock selection to portfolio evaluation. It employs a robust stock selection algorithm that ranks assets by synthesizing technical, fundamental, and advanced machine learning models. The core of the system is a sophisticated asset management engine that combines Mean-Variance Optimization with the Black-Litterman and Fama-French models to construct optimal, risk-managed portfolios based on both market data and specific investor insights. The entire framework is validated through a flexible backtesting engine, allowing for comprehensive performance analysis under various historical market conditions to ensure strategy viability. This project report goes over the important theory learned throughout the project along with a synopsis of the final implementation and results of the project.

Table of Contents

List of Figures

List of Tables

1 Technical Indicators

Definition 1: Candlestick Charts

Candlestick charts are widely used in technical analysis to represent price movement of an asset within a specified time period. Each candlestick contains information about the opening, closing, high, and low prices.

Components:

Each candlestick shows:

- **Open:** Price at the beginning of the time period.
- **High:** Highest price during the time period.
- **Low:** Lowest price during the time period.
- **Close:** Price at the end of the time period.

Color Interpretation:

- **Green Candlestick (Bullish):** Closing price $>$ opening price.
- **Red Candlestick (Bearish):** Closing price $<$ opening price.

Candlestick patterns help traders predict potential market movements.

Definition 2: Moving Averages

Moving Averages (MAs) are used to smooth out price fluctuations and identify the direction of trends over time.

Simple Moving Average (SMA):

The SMA is the arithmetic mean of closing prices over a period n:

$$SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Significance:

SMA helps in identifying support/resistance and trend directions. Crossovers signal buy/sell opportunities.

Exponential Moving Average (EMA) EMA gives more weight to recent prices:

$$EMA_t = \alpha P_t + (1 - \alpha) EMA_{t-1} \quad \text{where } \alpha = \frac{2}{n+1}$$

Significance EMA reacts faster to recent changes and is useful for short-term trading signals.

Definition 3: Relative Strength Index

RSI is a momentum oscillator measuring the speed and magnitude of price changes.

$$RS = \frac{\text{Average Gain over } n}{\text{Average Loss over } n}, \quad RSI = 100 - \frac{100}{1 + RS}$$

Typically $n = 14$.

Significance:

- **RSI > 70:** Overbought (possible correction).
- **RSI < 30:** Oversold (possible rebound).

RSI helps identify overextended conditions and potential reversals.

Definition 4: MACD

MACD is a trend-following indicator showing the relationship between two EMAs:

$$MACD = EMA_{12}(P) - EMA_{26}(P)$$
$$\text{Signal Line} = EMA_9(MACD)$$
$$\text{Histogram} = MACD - \text{Signal Line}$$

Significance

- **MACD > Signal Line:** Bullish signal.
- **MACD < Signal Line:** Bearish signal.
- **MACD crosses zero:** Trend change.

Definition 5: Bollinger Bands

Bollinger Bands consist of a central SMA and two bands at $\pm k$ standard deviations:

$$\text{Middle Band} = SMA_n(P)$$
$$\text{Upper Band} = SMA_n(P) + k \cdot \sigma$$
$$\text{Lower Band} = SMA_n(P) - k \cdot \sigma$$

Where $k = 2$ typically.

Significance

- **Price near upper band:** Overbought.
- **Price near lower band:** Oversold.
- **Bands widen:** Increased volatility.
- **Bands narrow:** Possible breakout.

Definition 6: ATR

ATR measures market volatility using the True Range (TR):

$$TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|)$$

$$ATR_t = \frac{1}{n}\sum_{i=0}^{n-1} TR_{t-i} \quad \text{or} \quad ATR_t = \frac{ATR_{t-1}(n-1) + TR_t}{n}$$

Significance

- **High ATR:** High volatility.
- **Low ATR:** Market consolidation.

ATR helps with setting stop-losses and position sizing.

2 Risk Management Strategies

Risk management forms the fundamental pillar of our portfolio strategy, focusing on balancing potential returns with acceptable risk levels across various market conditions.

Definition 7: Static Risk Controls

Stop Loss: Automatically closes positions at predetermined unfavorable prices to cap losses
Take Profit: Secures gains at specified favorable price levels

For long trades, Stop Loss is placed below entry price and Take Profit above; for short trades, positions are reversed. While effective for basic risk mitigation, these static levels can be susceptible to sudden market volatility.

2.1 Advanced Dynamic Strategies

2.1.1 Trailing Stop Loss for Long Trades

The Trailing Stop Loss dynamically adjusts stop-loss levels as asset prices increase, locking in profits while limiting downside risk.

Example Implementation:

Initial entry at \$100 with 6% trailing stop:

- Initial stop loss: \$94
- Price rises to \$110 $\rightarrow$ Stop loss moves to \$103.40

- Exit triggered if price drops below updated level

2.1.2 Dynamic Exit Condition for Short Trades

Traditional trailing stops prove less effective for short trades. Our Dynamic Exit Condition addresses this:

- Initial portfolio value: x dollars
- Exit threshold: $1.06x$
- If value drops to $0.5x$, threshold adjusts to $0.53x$
- Captured 10.89% return in simulated trades

2.2 Market Volatility-Based Adjustments

Recognizing external factors' impact on market volatility, particularly in BTC-USDT markets, we introduced multipliers to scale Stop Loss percentages during high-volatility periods when global stock markets are open.

3 Backtesting

3.1 Introduction to Backtesting

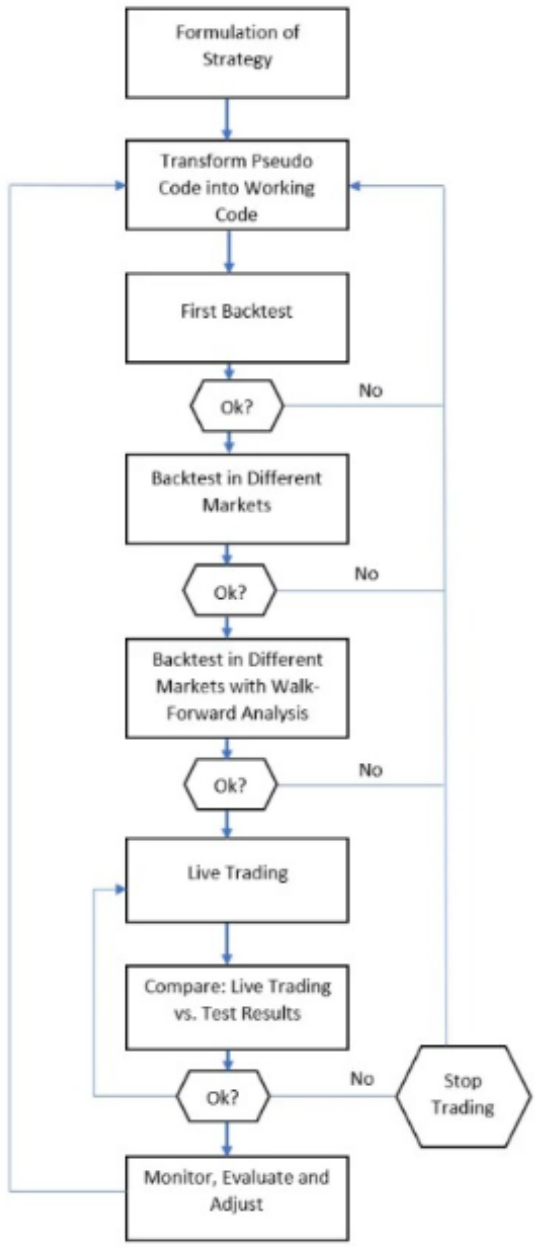


Figure 1: *An overview of the backtesting process*

What is backtesting and why do we need it?

Backtesting is a fundamental technique in algorithmic trading that involves testing a trading strategy on historical market data to evaluate its effectiveness and profitability. By simulating how a strategy would have performed in the past, traders can gain insight into its potential performance in the future without risking actual capital.

The goal of backtesting is to assess whether a strategy is robust, consistent, and capable of generating desirable returns while managing risks. It allows traders to identify flaws, optimize parameters, and avoid deploying unprofitable strategies in live markets. A successful backtest does not guarantee future success, but it provides valuable insights into the viability of a strategy. Poor backtest results can indicate that a strategy may be flawed, overly risky, or overly optimized. Backtesting also enables traders to compare multiple strategies, optimize parameters, and avoid impulsive decisions based on real-time experience.

3.2 How Does Backtesting Work?

Backtesting means testing a trading strategy on past market data to see how it would have performed. We take historical prices and apply our strategy's rules to decide when we would have bought or sold. At each step, we check if a trade should happen, update our virtual money or holdings, and keep track of how the strategy grows or loses money over time. This helps us understand if the strategy is profitable and what kind of risks it carries, without using real money.

Required Components

- Historical Data: Typically includes OHLC prices and trading volume.
- Trading Strategy Logic: Rules for entering and exiting trades.
- Performance Tracker: Measures profit/loss and tracks other metrics.

Walkthrough of a Backtest A backtest works by looping through historical data and executing trades when conditions are met. Here's a basic flow:

- Start with initial capital (e.g., $10,000).
- At each time step:
  - Check for a buy/sell signal based on strategy logic. (eg. if the MACD line crosses over the signal line, we get a "buy" signal.)
  - Execute a virtual trade if conditions are satisfied.(use all the cash to buy the BTC and calculate the portfolio value and remains based on the type of trade)
  - Update portfolio value. (portfolio value = cash + (holdings * current price))
- At the end, calculate KPIs like return, Sharpe ratio, and drawdown.(eg. if the sharpe ratio is higher, better the trade).

Code Snippet

```
initial_capital = 10000
cash = initial_capital
position = 0
portfolio = []

for i in range(1, len(data)):
    price = data['Close'].iloc[i]
    signal = data['signals'].iloc[i]

    # Buy Signal
    if signal == 1 and cash >= price:
        position = cash / price
        cash = 0
    # Sell Signal
    elif signal == -1 and position > 0:
        cash = position * price
        position = 0

    # Portfolio Value = Cash + Value of Held Position
    portfolio_value = cash + position * price
    portfolio.append(portfolio_value)

data['Portfolio_Value'] = portfolio
```

3.3 Risk Management in Backtesting

Risk management is a crucial component of any trading strategy and must be included during backtesting to ensure realistic performance analysis. Two widely used techniques are **Static Stop Loss/Take Profit** and **Trailing Stop Loss/Take Profit**. These mechanisms define when a trade should be exited, either to limit losses or lock in profits.

Static Stop Loss and Take Profit

A *static stop loss* and *take profit* are predefined price levels set when a trade is entered. These levels remain fixed throughout the trade's duration.

- **Stop Loss:** Closes the trade if the price moves unfavorably beyond a set threshold to limit losses.
- **Take Profit:** Exits the trade at a predefined profit level to secure gains.

This method is simple, consistent, and well-suited for strategies that follow fixed **risk-reward ratios**.

Example (Long Position):

- Entry Price: $100
- Static Stop Loss: $95 (i.e., a $5 risk)
- Static Take Profit: $110 (i.e., a $10 profit target)

In this setup, the trade will automatically exit at $95 or $110 depending on which level is hit first, regardless of price movement in between.

Trailing Stop Loss and Take Profit

A *trailing stop loss* and *take profit* are dynamic risk-management tools that adjust in real time as the market price moves favorably.

- **Trailing Stop Loss:** Follows the market price at a fixed distance, locking in profits as the price rises.
- **Trailing Take Profit:** (less commonly used) shifts the profit target upward to capture additional gains in a trend.

If the price reverses by the trailing distance, the trade exits automatically. This method is ideal for **trend-following strategies**.

Example (Long Position with 5% Trailing Stop Loss):

- Entry Price: $100
- Price rises to $110 $\rightarrow$ Trailing Stop Loss moves to $104.5 (5% below peak)
- If price drops to $104.5 $\rightarrow$ Trade exits

This dynamic approach helps protect gains while allowing profitable trades to continue running in strong trends.

3.4 Key Performance Indicators (KPIs)

Total Return Definition: Total return tells us how much profit or loss the strategy made from the starting point to the end.

Formula:

$$\text{Total Return} = \left(\frac{\text{Final Portfolio Value} - \text{Initial Capital}}{\text{Initial Capital}} \right) \times 100$$

How to Interpret:

Return type	Interpretation
Positive	Strategy made a profit
Negative	Strategy lost money
Higher return	More profitable — but may involve more risk
Consistent Return	More stable, often preferred with lower risk

Table 1: *Interpreting Return Types*

Code Snippet

Code Example:

```
initial_value = 10000
final_value = data['Portfolio_Value'].iloc[-1]
total_return = (final_value - initial_value) / initial_value * 100
print("Total Return:", total_return, "%")
```

Sharpe Ratio Definition: The Sharpe Ratio tells us how much extra return a strategy is giving for every unit of risk it is taking.

Imagine two person A and B invest in different strategies:

- Person A makes 12% return per year, but their returns jump up and down a lot.
- Person B makes 10% return, but with very steady performance.

Even though A's return is higher, your friend might have a better Sharpe Ratio — because their strategy is more consistent and less risky.

The Sharpe Ratio helps compare which strategy is truly "better," not just in terms of how much it made, but also how safely it made it

Formula:

$$\text{Sharpe Ratio} = \frac{\text{Average Return} - \text{Risk-Free Return}}{\text{Standard Deviation of Return}}$$

Average Return: How much profit the strategy makes on average.

Risk-Free Return: The return you could earn without taking any risk at all. Think of it like keeping your money in a super-safe place, like a government savings bond or fixed deposit, where you're almost guaranteed to get your money back with a small amount of interest.

Standard Deviation: How much the returns fluctuate. More fluctuation means more risk. If the standard deviation is close to the average return, it implies low fluctuation. If it is far from the average, it implies high fluctuation.

How to Interpret:

Sharpe Ratio	Interpretation
< 1	Poor – return is not worth the risk taken
1 to 2	Acceptable – okay for some strategies
2 to 3	Good – the strategy has solid performance
> 3	Excellent – very strong risk-adjusted return

Table 2: *Sharpe Ratio Interpretation Guide*

Code Snippet

Code Example:

```
import numpy as np
data['Daily_Returns'] = data['Portfolio_Value'].pct_change().dropna()
risk_free_rate = 0.0001
excess_returns = data['Daily_Returns'] - risk_free_rate
sharpe_ratio = excess_returns.mean() / excess_returns.std()
annualized_sharpe = sharpe_ratio * np.sqrt(252)
```

Maximum Drawdown Definition: Maximum Drawdown (MDD) tells us the worst fall in our portfolio value from a high point (peak) to a low point (trough) before it recovers. It represents the largest temporary loss during the backtest period. This is important because it shows the risk of deep losses, even if the strategy looks profitable overall. A strategy that makes good returns but also suffers large drops can be dangerous in real-world trading.

Formulas:

$$\text{Max Drawdown} = \max\left(\frac{\text{Peak} - \text{Trough}}{\text{Peak}}\right) \times 100$$

How to Interpret:

Maximum Drawdown	Interpretation
$< 30\%$	Very safe and stable
10% to 30%	Moderate risk
More than 30%	High risk — big temporary losses

Table 3: *Drawdown Risk Interpretation*

Maximum Dip Definition: Maximum Dip measures the biggest loss from the entry point of a trade to the lowest point during the trade. It tells you how bad the trade got after entering, regardless of whether it recovered or not. It focuses on entry risk — the amount of pain a trader would have felt after placing a trade.

Formulas:

$$\text{Max Dip} = \max\left(\frac{\text{Entry Price} - \text{Lowest Price}}{\text{Entry Price}}\right) \times 100$$

How to Interpret:

Maximum Dip	Interpretations
$< 5\%$	Low dip — safe trade
5%–20%	Moderate dip — normal fluctuations
$> 20\%$	High dip — risky or badly timed entry

Table 4: *Interpretation of Maximum Dip*

Code Snippet

```
a = 0
max__drawdown = []
max__dip = []

for i in range(0, ((int(len(trades) / 2)) * 2), 2):
    drawdown = []
    index1 = trades[i]
    index2 = trades[i + 1]
    stocks = num_stocks[a]
    remain = remains[a]

    max1 = dd1['Portfolio_Value'].iloc[index1]
    min1 = dd1['Portfolio_Value'].iloc[index1]

    for j in range(index1, index2 + 1):
        portfolio = dd1['Portfolio_Value'].iloc[j]
        max1 = max(max1, portfolio)
        min1 = min(min1, portfolio)
        drawdown_percent = ((max1 - portfolio) / portfolio) * 100
        drawdown.append(drawdown_percent)

    max__drawdown.append(max(drawdown))
    dip_from_entry = ((dd1['Portfolio_Value'].iloc[index1] - min1) /
                      dd1['Portfolio_Value'].iloc[index1]) * 100
    max__dip.append(dip_from_entry)
    a += 1

twd1['Max Drawdown for Trade'] = max__drawdown
twd1['Max Dip for Trade'] = max__dip
```

3.5 Benefits and Limitations of Backtesting

Benefits

- **Risk-free testing:** Allows you to test strategies without using real money.
- **Strategy refinement:** Helps tweak entry/exit rules and parameters to improve performance.
- **Early problem detection:** Identifies weaknesses or risky behaviors in your strategy.
- **Performance comparison:** Lets you objectively compare different strategies using metrics.

Limitations

- **Overfitting risk:** Strategy may be too closely tailored to past data and fail in the future.
- **No guarantee of future success:** Past performance does not always reflect future market behavior.
- **Assumption bias:** Unrealistic assumptions (like perfect execution, zero fees) can mislead results.
- **Bad or incomplete data:** Poor-quality historical data can give inaccurate outcomes.

Backtesting is a crucial first step in developing and evaluating a trading strategy. It allows traders to simulate trades on historical data and measure performance using objective metrics like return, drawdown, and Sharpe Ratio.

It helps in refining strategies, identifying weaknesses, and avoiding risky or unprofitable setups before risking real capital. However, backtesting is not enough on its own. Market conditions change, and strategies that perform well in the past may fail in the future.

4 Fama-French Three-Factor Model

The Fama-French Three-Factor Model enhances traditional asset pricing by incorporating additional risk factors beyond market risk, providing a comprehensive framework for analyzing stock returns within our project. Developed by Eugene Fama and Kenneth French, this model extends the Capital Asset Pricing Model (CAPM) by addressing empirical anomalies in stock performance, making it a pivotal tool in our portfolio management strategies.

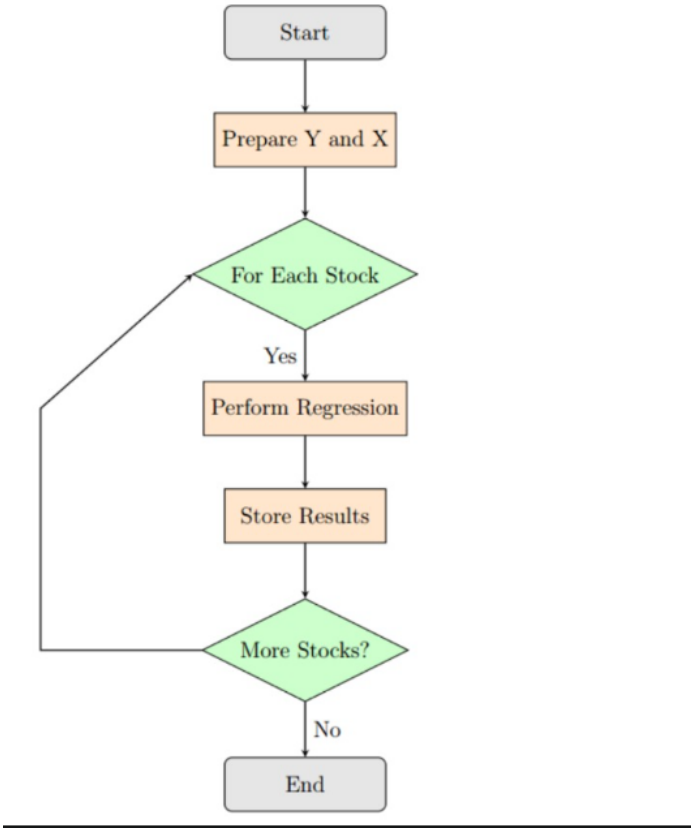


Figure 2: *An overview of the Fama French Model*

4.1 Enhancement Over CAPM

The CAPM focuses solely on market risk to predict expected returns, using the formula:

$$E(R_i) = R_f + \beta_i \cdot (E(R_m) - R_f)$$

Where:

- $E(R_i)$: Expected return on the asset
- R_f: Risk-free rate
- β_i: Beta coefficient of market sensitivity
- $E(R_m) - R_f$: Market risk premium

However, CAPM often underestimates returns for small-cap and value stocks. The Fama-French model addresses this limitation by adding two factors—size and value—to explain variations in returns more effectively:

$$R_p - R_f = \alpha + \beta_m(R_m - R_f) + \beta_{smb} \cdot SMB + \beta_{hml} \cdot HML$$

Where:

- R_p: Return of the portfolio or asset
- R_f: Risk-free rate
- R_m: Market return
- SMB: Small Minus Big, capturing size effect
- HML: High Minus Low, capturing value effect
- α: Abnormal return not explained by factors
- $\beta_m, \beta_{smb}, \beta_{hml}$: Sensitivities to respective factors

4.2 Key Factors and Their Construction

[Core Components of the Model] The Fama-French Three-Factor Model incorporates three distinct factors to describe stock returns:

- **Market Risk Premium ($R_m - R_f$)**: Represents the excess return of the market over the risk-free rate, akin to CAPM, reflecting general market exposure.
- **Size Factor (SMB - Small Minus Big)**: Captures the historical outperformance of small-cap companies over large-cap companies, calculated as the difference between average returns of small-cap and large-cap portfolios.
- **Value Factor (HML - High Minus Low)**: Reflects the tendency of value stocks (high book-to-market ratio) to outperform growth stocks (low book-to-market ratio), computed as the difference between average returns of high and low book-to-market portfolios.

4.3 Empirical Evidence and Theoretical Insights

[Historical Performance Insights] Empirical data analyzed during our project confirms key observations:

- Small-cap stocks consistently yield higher returns accompanied by increased risk compared to large-cap stocks.
- Value stocks with high book-to-market ratios outperform growth stocks over extended periods, indicating a value premium.
- The model explains over 90% of diversified portfolio returns, a significant improvement over CAPM's 70% within sample data.

The model's ability to account for cross-sectional variations in stock returns stems from its recognition of size and value as systematic risk factors, beyond mere market risk. This

aligns with findings that higher returns are associated with higher risk in these dimensions, supporting the efficient market hypothesis to an extent, though anomalies remain.

4.4 Limitations and Critiques

Despite its strengths, the Fama-French Three-Factor Model has notable limitations:

- **Missing Anomalies:** It does not capture other documented effects such as momentum, where past winners continue to outperform, leading to extensions like the Carhart Four-Factor Model.
- **Data Mining Concerns:** Critics argue that size and value factors may result from overfitting historical data, questioning their persistence in future or different market contexts.
- **Theoretical Ambiguity:** While market risk has a clear basis in CAPM, the theoretical underpinnings of size and value premiums are less defined, with explanations ranging from financial distress to behavioral biases.
- **Country-Specific Factors:** The factors are often country-specific, performing better with local rather than global data, as observed in markets like the U.S., Canada, Japan, and the U.K.

4.5 Application in Portfolio Management

In our project, the Fama-French Three-Factor Model was applied to enhance portfolio construction and risk assessment. Using regression analysis on historical data for selected stocks, we estimated factor sensitivities ($\beta_m, \beta_{smb}, \beta_{hml}$) to understand how market, size, and value risks impact expected returns. This approach guided asset selection by tilting portfolios towards small-cap or value stocks when favorable premiums were identified, aligning with our goal of optimizing risk-adjusted returns.

[Practical Implementation] Key applications in our project included:

- **Risk Assessment:** Determining portfolio sensitivity to size and value factors alongside market risk for comprehensive risk profiling.
- **Performance Attribution:** Decomposing portfolio returns to identify contributions from each factor, aiding in understanding outperformance or underperformance sources.
- **Strategic Allocation:** Adjusting portfolio weights to exploit historical premiums in small-cap and value stocks, based on empirical factor loadings.

4.6 Conclusion

The Fama-French Three-Factor Model served as a critical framework in our project, offering a more robust explanation of stock returns compared to CAPM by incorporating size and value factors. Its empirical success in explaining over 90% of portfolio return variations provided valuable insights for portfolio optimization. However, limitations such as the exclusion of momentum and theoretical ambiguities were acknowledged, prompting integration with other models like Black-Litterman for enhanced applicability. This model remains a cornerstone of our asset pricing analysis, balancing empirical rigor with practical utility in financial decision-making.

5 Markowitz Mean Variance Optimization

Markowitz's MVO underpins modern portfolio theory, guiding our approach to constructing optimal portfolios by balancing risk and return effectively.It provides a quantitative framework for constructing portfolios that maximize expected return for a given level of risk (variance), or equivalently, minimize risk for a given level of return.

5.1 Key Components

[Key Components] **Expected Returns:** Weighted average of asset returns Given a portfolio of n assets with weights $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$, and expected returns $\boldsymbol{\mu} = [\mu_1, \mu_2, \ldots, \mu_n]^T$, the expected return of the portfolio is:

$$\mathbb{E}[R_p] = \mathbf{w}^T \boldsymbol{\mu}$$

Risk: Measured as portfolio variance or standard deviation Let Σ be the $n \times n$ covariance matrix of asset returns. The portfolio variance is:

$$\sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w}$$

Covariance: In Markowitz Mean-Variance Optimization, the covariance matrix Σ captures the pairwise covariances between asset returns in a portfolio. It is defined as:

$$\Sigma = \begin{bmatrix} \text{Var}(R_1) & \text{Cov}(R_1, R_2) & \cdots & \text{Cov}(R_1, R_n) \\ \text{Cov}(R_2, R_1) & \text{Var}(R_2) & \cdots & \text{Cov}(R_2, R_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(R_n, R_1) & \text{Cov}(R_n, R_2) & \cdots & \text{Var}(R_n) \end{bmatrix}$$

where:

$$\text{Cov}(R_i, R_j) = \mathbb{E}\left[(R_i - \mu_i)(R_j - \mu_j)\right]$$

Let $\mathbf{R} = [R_1, R_2, \ldots, R_n]^T$ denote the random return vector and $\boldsymbol{\mu} = \mathbb{E}[\mathbf{R}]$ the expected return vector, then the covariance matrix can be compactly expressed as:

$$\Sigma = \mathbb{E}[(\mathbf{R} - \boldsymbol{\mu})(\mathbf{R} - \boldsymbol{\mu})^T]$$

Efficient Frontier: Optimal risk-return combinations which we get on solving the optimization problem of MVO.

5.2 Objective

The standard form of the Markowitz optimization problem is to optimize:

$$\min_{\mathbf{w}} \quad \mathbf{w}^T \Sigma \mathbf{w}$$

subject to:

$$\mathbf{w}^T \boldsymbol{\mu} = \mu_p \quad \text{(target return)}$$

$$\sum_{i=1}^{n} w_i = 1 \quad \text{(fully invested portfolio)}$$

In the case of short selling we introduce another constraint as:

$$w_i \geq 0 \quad \forall i$$

5.3 Efficient Frontier

The set of optimal portfolios forms the efficient frontier — the boundary of the minimum variance for a given return.

Definition 8: Efficient Frontier

Portfolios on the frontier are optimal. Any portfolio below the frontier is inefficient (higher risk for the same return).

5.4 Advantages and Limitations

Advantages

- Provides a quantitative, theoretically grounded approach to diversification
- Forms the basis of modern portfolio theory and CAPM

Limitations

- Assumes normally distributed returns and stable covariances
- Sensitive to input estimation errors
- No guarantee of outperformance out-of-sample

We address these through integrated models and sensitivity analyses.

6 Black- Litterman Model

The Black-Litterman Model provides a sophisticated framework for portfolio optimization, blending market equilibrium with subjective investor views.

The key innovation of the Black–Litterman model lies in its ability to systematically blend market consensus with individual investor insights, creating more stable and realistic portfolio allocations.

6.1 Investor Views

The key innovation of the Black–Litterman model is its ability to incorporate subjective views of the investor about future returns. These views are usually related to specific assets or asset classes and may involve an expected return different from the market equilibrium return.

These views can be categorized as:

1. **Absolute views**: An investor believes a particular asset will have a return of 10%
2. **Relative views**: An investor believes Asset A will outperform Asset B by 5%

6.2 Bayesian Framework

The model applies Bayesian inference to combine the CAPM equilibrium returns (prior) with the investor's views (new information) to generate a posterior distribution of expected returns.

This results in adjusted returns that account for both the market equilibrium and the investor's subjective expectations, weighted by the investor's confidence in their views.

The Bayesian update can be expressed as:

$$\text{Posterior} \propto \text{Prior} \times \text{Likelihood} \tag{1}$$

6.3 Reverse Optimization

Instead of starting with expected returns (as in traditional mean-variance optimization), the Black–Litterman model starts by inferring the expected returns implied by the market (market equilibrium returns) using the market capitalization weights.

This process is called *reverse optimization*, where the equilibrium returns are derived by assuming that the market portfolio is mean-variance efficient.

The implied equilibrium returns are calculated as:

$$\boldsymbol{\mu} = \delta \boldsymbol{\Sigma} \boldsymbol{w}_{mkt} \tag{2}$$

where:

- $\boldsymbol{\mu}$ = Vector of implied equilibrium returns
- δ = Risk aversion coefficient
- $\boldsymbol{\Sigma}$ = Covariance matrix of asset returns
- $\boldsymbol{w}_{mkt}$ = Market capitalization weights

The key innovation of the Black–Litterman model is its ability to incorporate subjective views of the investor about future returns. These views are usually related to specific assets or asset classes and may involve an expected return different from the market equilibrium return.

These views can be categorized as:

1. **Absolute views**: An investor believes a particular asset will have a return of 10%
2. **Relative views**: An investor believes Asset A will outperform Asset B by 5%

6.4 Mathematical Foundation

Equilibrium Returns:

$$\Pi = \lambda \Sigma W_m$$

Posterior Returns:

$$\mu_{BL} = \left[(\tau\Sigma)^{-1} + P^T\Omega^{-1}P\right]^{-1}\left[(\tau\Sigma)^{-1}\Pi + P^T\Omega^{-1}Q\right]$$

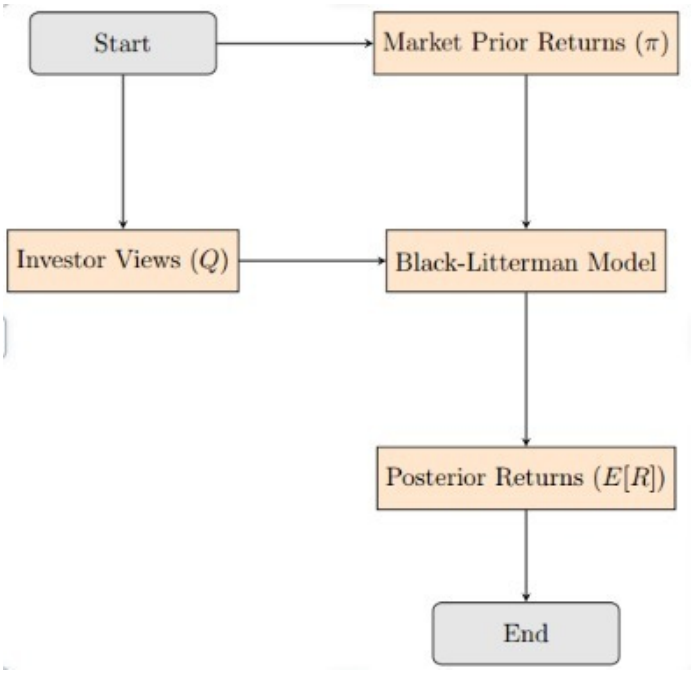
Where:

$$\Pi = \text{Equilibrium excess returns} \tag{3}$$

$$\lambda = \text{Risk aversion coefficient} \tag{4}$$

$$\Sigma = \text{Covariance matrix} \tag{5}$$

$$W_m = \text{Market capitalization weights} \tag{6}$$



Theorem 9: Steps of the Black-Litterman Model

Step 1: Market Equilibrium
Use the global market portfolio and covariance matrix to compute the implied excess equilibrium returns. These are based on market capitalization weights and represent the market's expected returns absent any special views.

Step 2: Formulate Views
The investor introduces views on assets, either absolute (e.g., "Asset X will return 7

$$\boldsymbol{P\mu} = \boldsymbol{Q} + \boldsymbol{\epsilon} \tag{7}$$

where $\boldsymbol{P}$ is the picking matrix, $\boldsymbol{Q}$ is the view vector, and $\boldsymbol{\epsilon}$ is the error vector with covariance $\boldsymbol{\Omega}$.

Step 3: Blend Views and Equilibrium Returns
Investor views are combined with market equilibrium returns using Bayesian techniques. The model adjusts equilibrium returns based on the investor's confidence; high confidence gives views more influence, while low confidence defaults toward the market equilibrium.

Step 4: Posterior Returns
The output is a new set of expected returns, called *posterior returns*, which blend market equilibrium with the investor's views. The posterior expected returns are calculated as:

$$\boldsymbol{\mu}_{BL} = \boldsymbol{\mu} + \boldsymbol{\Sigma P}^T(\boldsymbol{P\Sigma P}^T + \boldsymbol{\Omega})^{-1}(\boldsymbol{Q} - \boldsymbol{P\mu}) \tag{8}$$

And the posterior covariance matrix is:

$$\boldsymbol{\Sigma}_{BL} = \boldsymbol{\Sigma} - \boldsymbol{\Sigma P}^T(\boldsymbol{P\Sigma P}^T + \boldsymbol{\Omega})^{-1}\boldsymbol{P\Sigma} \tag{9}$$

Step 5: Optimize Portfolio
Posterior returns are used in a mean-variance optimization framework to construct the final portfolio. The resulting allocation is adjusted for the investor's views but remains anchored to the efficient frontier. The optimal weights are:

$$\boldsymbol{w}^* = \frac{1}{\delta}\boldsymbol{\Sigma}_{BL}^{-1}\boldsymbol{\mu}_{BL} \tag{10}$$

6.5 Advantages of the Black–Litterman Model

Stability:
Traditional mean-variance optimization is highly sensitive to small changes in expected returns, often leading to extreme and unrealistic portfolio weights. The Black–Litterman model smooths out this sensitivity by anchoring on market equilibrium returns.

Incorporates Investor Views:
It allows investors to systematically incorporate their views into the portfolio optimization process, giving them more control over the outcome without completely abandoning the market information.

Reduced Estimation Error:
Traditional models rely on historical returns to estimate expected returns, which can be error-prone. The Black–Litterman model reduces estimation errors by incorporating market equilibrium information, which is generally considered more reliable.
Balanced Portfolio:
The model avoids extreme portfolios, providing more reasonable asset allocations by maintaining a balance between market consensus and individual views.

The Black–Litterman model is a powerful tool for portfolio construction, blending market equilibrium with individual investor views in a systematic and coherent way. It improves the stability of portfolio optimization by relying on a solid foundation of market data, while still allowing flexibility for subjective opinions on expected returns.

7 Exploratory Data Analysis and Feature Selection

Exploratory Data Analysis (EDA) is a crucial aspect of data science that helps in understanding the underlying structure of the data. It involves analyzing and summarizing data visually and statistically to uncover patterns or relationships. The following subsections guide through all essential steps in conducting thorough EDA.

7.1 Data Collection and Cleaning

Before any analytical step in the pipeline, the research question must be clearly defined and outlined. Subsequently, one can identify the required data to solve that research question and obtain it from various sources such as databases, websites, APIs, ensuring the data is collected in an accessible and manipulable format.

Before performing Exploratory Data Analysis on the data, we need to import the required libraries for analysis. Most data analysis in this project utilizes Python with the following essential libraries:

Essential Python Libraries:

- `pandas` – Data manipulation and analysis
- `numpy` – Numerical computing
- `matplotlib.pyplot` – Data visualization
- `seaborn` – Statistical data visualization
- `scipy.stats` – Statistical functions

Data cleaning ensures high-quality data for further inspection and analysis. The essential steps for effective EDA include:

1. **Identify and Remove Duplicates** – Eliminate redundant observations that could bias results
2. **Handle Missing Values** – Apply appropriate imputation strategies based on missing data patterns
3. **Address Outliers** – Identify and treat outliers using statistical methods or domain knowledge
4. **Remove Irrelevant Data** – Filter out data that doesn't contribute to the research objectives
5. **Handle Data Inconsistencies** – Standardize formats, correct errors, and ensure data integrity

7.2 Data Exploration and Analysis

After collecting and cleaning the data the next step is to explore the data and find the trends and patterns in the data. Exploring data generally involves these 4 steps:
Understanding the Central Tendencies of the Data:★ Calculating central tendencies of the data like mean, median and mode help understand the data and there are inbuilt functions to calculate either using pandas or in numpy per column.
Visualising the Data:★There are multiple ways to visualize the data depending on the type of the data and the objective of the visualization. A simple line graph helps visualize the raw data point with respect to some indexing set and a line chart helps visualise linear relationships in the data by plotting a line graph between them. A box plot helps visualize the distribution of the data with respect to some indexing set, and a histogram.
Examining Correlations★ We define correlation as the tendency for 2 variables to change together. Mathematically we use correlation coefficients to quantify the correlation as these coefficients range from -1 to 1 and quantify the strength and direction of the trend. We can use a correlation matrix to see the correlation coefficients for all possible pairs of variables. To visualise the correlation between any two variables we can use a scatter plot to see how they move with each other.
Hypothesis Testing★ Hypothesis Testing is a statistical method used to make inferences about a population from a sample. It allows us to test our claims and thus form further insights into our data generate new or modified hypotheses. The steps involved in are:

1. **Formulate the Null and Alternative Hypothesis**
2. **Choose a statistical Test**
3. **Set a significance level(alpha)**

4. **Calculate the test statistics and p value**
5. **Make a decision, draw conclusions and make new hypotheses.**

7.3 Feature Selection

Feature selection is the process of identifying and choosing the most relevant variables (or **features**) from your dataset to build a predictive model. During Exploratory Data Analysis (EDA), this helps you understand your data's structure and focus on the most promising features for modeling.In our project we layed emphasis on three main feature selection processes namely; PCA, Mutual Information and Random Forests.

7.3.1 Principal Component Analysis (PCA)

PCA isn't technically a feature **selection** method; it's a feature **extraction** method. It reduces dimensionality by transforming your original features into a new, smaller set of uncorrelated variables called **principal components**.

- **How it Works in EDA:** During EDA, you use PCA to see how much of your data's variance can be captured by just a few components. You would create a **scree plot** to visualize the variance explained by each component. The "elbow" of the plot suggests the optimal number of components to retain.
- **Analogy:** Think of it like summarizing a complex meal. Instead of listing every single ingredient (original features), you describe it by its main flavor profiles like "savory," "spicy," and "sweet" (principal components).
- **Key Takeaway:** Use PCA when you have many correlated features and want to reduce dimensionality, but be aware that the resulting components are less interpretable than the original features.

7.3.2 Random Forests (Feature Importance)

This is a popular **embedded method**, where the machine learning model itself helps select the features.

- **How it Works in EDA:** You train a baseline Random Forest model on your data. The model then calculates an **importance score** for each feature based on how much it contributes to making accurate predictions across all the decision trees in the forest. You can then visualize these scores in a bar chart to rank your features.
- **Key Takeaway:** This is a powerful and intuitive method to rank the predictive power of your original features. It's excellent because it captures complex, non-linear relationships between features and the target.

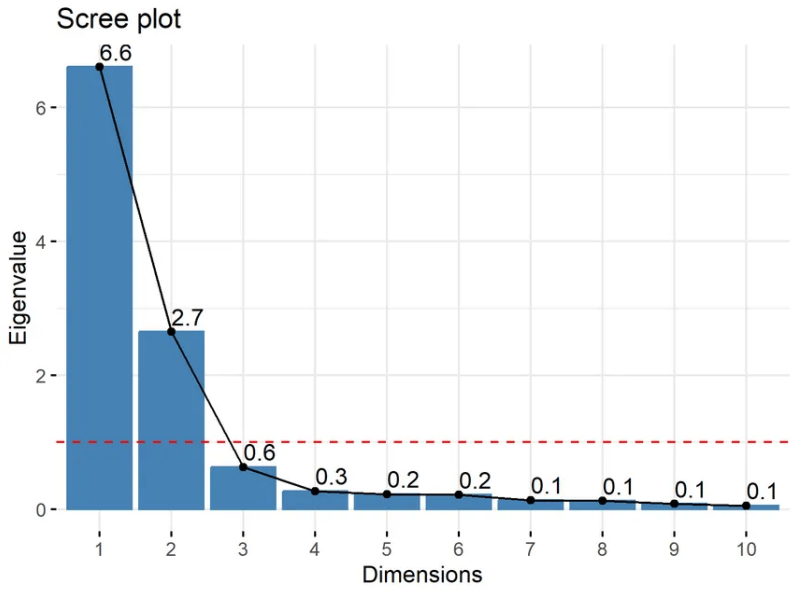


Figure 3: *A PCA scree plot showing the variance explained by each principal component. The "elbow" indicates a point of diminishing returns for adding more components.*

```
In [148]: plt.title('Feature Importance')
          plt.bar(range(X_train.shape[1]), importances[sorted_indices], align='center')
          plt.xticks(range(X_train.shape[1]), X_train.columns[sorted_indices], rotation=90)
          plt.tight_layout()
          plt.show()
```

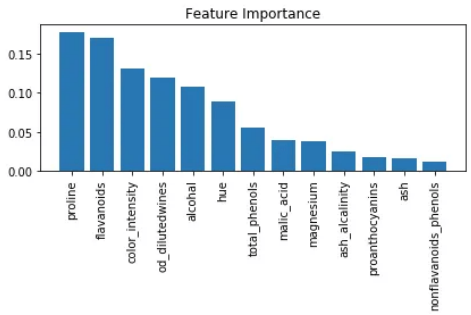


Figure 4: *A bar chart showing feature importance scores from a Random Forest model. This provides a clear ranking of the most predictive features.*

7.3.3 Mutual Information

Mutual Information is a **filter method** that measures the statistical dependency between two variables. It's more powerful than simple correlation because it can capture any kind of relationship, not just linear ones.

- **How it Works in EDA:** You calculate the mutual information score between each feature and the target variable. A higher score means the feature shares more information with the target, making it a stronger candidate for your model.
- **vs. Correlation:** If the relationship between a feature and the target is U-shaped, the correlation coefficient would be close to zero, suggesting no relationship. Mutual

Information, however, would detect this non-linear dependency and assign a high score.

- **Key Takeaway:** Use mutual information to rank features based on the strength of their relationship (both linear and non-linear) with the target variable. It's a robust statistical approach that doesn't require training a model.

8 Decision Trees and Random Forests

Decision trees and random forests represent fundamental supervised learning algorithms used for both classification and regression tasks. While they share common principles, they differ significantly in their structure and approach to prediction.

8.1 Decision Trees

A **decision tree** is a hierarchical, flowchart-like structure that makes decisions by traversing from a root node down to leaf nodes. Each internal node represents a feature test, branches represent test outcomes, and leaf nodes contain final predictions.

Definition 10: Decision Tree Components

A decision tree consists of three fundamental node types:

- **Root Node:** The starting point representing the entire dataset
- **Internal Nodes:** Decision points based on feature values
- **Leaf Nodes:** Terminal nodes containing final predictions

Key Concepts and Metrics

The effectiveness of decision trees relies on several important concepts:

- **Splitting Criteria** – The process of dividing nodes to create homogeneous subsets
- **Impurity Measures** – Metrics to evaluate the quality of splits:
  - **Gini Impurity:** Measures misclassification probability

$$G = 1 - \sum_{i=1}^{n} p_i^2$$

where p_i represents the probability of class i in the node.

  - **Entropy and Information Gain:** Quantifies disorder reduction

$$H(S) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

Information gain for attribute A:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- **Pruning Techniques** – Methods to prevent overfitting through complexity control

The cost complexity pruning formula balances accuracy and simplicity:

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

where $R(T)$ represents misclassification error, $|T|$ denotes the number of leaves, and α controls the complexity penalty.

8.2 Random Forests

A **random forest** is an ensemble learning method that combines predictions from multiple decision trees, leveraging the wisdom of crowds principle to achieve superior performance and robustness.

Figure 5: *Conceptual representation of random forest ensemble learning*

Core Mechanisms

Random forests employ several key techniques to achieve superior performance:

- **Bootstrap Aggregating (Bagging)** – Creates diverse training subsets through sampling with replacement
- **Random Feature Selection** – At each node, considers only a random subset of features
- **Majority Voting/Averaging** – Combines individual tree predictions for final output
- **Feature Importance Ranking** – Quantifies variable contributions across all trees

8.3 Comparative Analysis

Table 5: *Comprehensive comparison of decision trees and random forests*

Characteristic	Decision Tree	Random Forest
Architecture	Single tree structure	Ensemble of multiple trees
Overfitting Risk	High susceptibility	Significantly reduced risk
Prediction Accuracy	Moderate performance	Superior accuracy and stability
Model Interpretability	Highly interpretable	Limited interpretability
Training Strategy	Uses complete dataset	Bootstrap sampling approach
Feature Selection	All features considered	Random feature subsets
Computational Cost	Low resource requirements	Higher computational demands
Robustness	Sensitive to data changes	Robust to noise and outliers

9 Boosting Algorithms

Boosting represents a powerful ensemble learning technique that sequentially combines multiple weak learners to create a robust, high-performance model. The fundamental principle involves iterative error correction, where each new model focuses on previously misclassified instances.

9.1 Fundamental Concepts

Definition 11: Boosting Terminology

Key boosting concepts include:

- **Weak Learners:** Simple models performing slightly better than random chance
- **Sequential Learning:** Models built iteratively, each focusing on previous errors
- **Instance Weighting:** Misclassified samples receive increased importance
- **Additive Modeling:** Final prediction combines all weak learner outputs

The mathematical foundation of boosting can be expressed as: $F(x) = \sum_{i=1}^{M} \alpha_i h_i(x)$ where $F(x)$ represents the final prediction, $h_i(x)$ denotes the i-th weak learner, and α_i represents its corresponding weight in the ensemble.

9.2 Gradient Boosting

Gradient Boosting employs a gradient descent approach to minimize loss functions. Rather than reweighting data points, each new weak learner predicts the **residuals** (errors) from previous models.

Algorithm 1 Gradient Boosting Algorithm

Input: Training dataset $\{(x_i, y_i)\}_{i=1}^n$, loss function $L(y, F(x))$, number of iterations M
Output: Final model $F_M(x)$

1: Initialize $F_0(x) = \arg\min_\gamma \sum_{i=1}^n L(y_i, \gamma)$
2: **for** $m = 1$ to M **do**
3: Compute residuals: $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$
4: Fit weak learner $h_m(x)$ to residuals $\{(x_i, r_{im})\}_{i=1}^n$
5: Find optimal step size: $\gamma_m = \arg\min_\gamma \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$
6: Update model: $F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$
7: **end for**

The iterative update formula can be written as: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$ where η represents the learning rate controlling step size.

9.3 XGBoost (eXtreme Gradient Boosting)

XGBoost represents an optimized, scalable gradient boosting implementation renowned for exceptional performance and computational efficiency.

Key Points

XGBoost incorporates several advanced features:

- **Regularization:** L1 and L2 penalties prevent overfitting
- **Parallel Processing:** Accelerates tree construction through parallelization
- **Missing Value Handling:** Intelligent strategies for incomplete data
- **Tree Pruning:** Both pre-pruning and post-pruning techniques
- **Cross-Validation:** Built-in validation for optimal hyperparameters

The XGBoost objective function combines prediction error with regularization: $\text{Objective}(\Theta) = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$ where L represents the loss function and $\Omega(f_k)$ penalizes model complexity.

9.4 CatBoost (Categorical Boosting)

CatBoost specializes in handling categorical features effectively without extensive preprocessing requirements.

Theorem 12: Ordered Boosting Principle

CatBoost employs ordered boosting to eliminate prediction shift bias through permutation-based target statistics, ensuring more robust categorical feature handling.

Key innovations of CatBoost include:

- **Ordered Boosting** – Prevents prediction shift through careful permutation strategies
- **Categorical Feature Processing** – Direct handling without manual encoding
- **Symmetric Trees** – Uniform split conditions at each tree level
- **GPU Acceleration** – Optimized implementation for high-performance computing

10 ARIMA Model

The **ARIMA (Autoregressive Integrated Moving Average)** model provides a comprehensive statistical framework for time series analysis and forecasting. ARIMA models are denoted as **ARIMA(p, d, q)**, representing three fundamental components.

10.1 ARIMA Components

Definition 13: ARIMA Parameters

The ARIMA(p, d, q) notation represents:

- **p:** Autoregressive order (number of lag observations)
- **d:** Degree of differencing (integration order)
- **q:** Moving average order (number of lagged forecast errors)

Autoregressive (AR) Component

The autoregressive component models current values as linear combinations of past observations: $Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \epsilon_t$

where:

- Y_t – Current time series value
- c – Intercept constant

- ϕ_i – Autoregressive parameters
- ϵ_t – White noise error term
- p – Number of lagged observations

Integrated (I) Component

The integration component addresses **non-stationarity** through differencing operations. Stationarity requires constant statistical properties over time.

Important

Non-stationary time series exhibit changing means, variances, or autocorrelation structures over time, violating ARIMA modeling assumptions. Differencing transforms non-stationary series into stationary ones suitable for analysis.

Moving Average (MA) Component

The moving average component incorporates past forecast errors: $Y_t = \mu + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + \cdots + \theta_q\epsilon_{t-q}$

where:

- μ – Series mean
- θ_i – Moving average parameters
- q – Number of lagged error terms

10.2 Complete ARIMA Model

The comprehensive ARIMA(p, d, q) model combines all components: $\Delta^d Y_t = c + \phi_1\Delta^d Y_{t-1} + \cdots + \phi_p\Delta^d Y_{t-p} + \epsilon_t + \theta_1\epsilon_{t-1} + \cdots + \theta_q\epsilon_{t-q}$

where $\Delta^d Y_t$ represents the series differenced d times.

10.3 Model Development Process

Algorithm 2 ARIMA Model Development Methodology

Input: Time series data $\{Y_t\}_{t=1}^{T}$
Output: Fitted ARIMA(p, d, q) model

1: **Stationarity Assessment:** Apply Augmented Dickey-Fuller test
2: **Differencing:** Apply differencing operations until stationarity achieved
3: **Parameter Identification:** Analyze ACF and PACF plots for p, q values
4: **Model Estimation:** Fit ARIMA model using maximum likelihood
5: **Diagnostic Checking:** Validate residuals for white noise properties
6: **Forecasting:** Generate future predictions with confidence intervals

11 Evaluation Metrics

Model evaluation is a critical step in machine learning to determine a model's performance. For classification problems, several key metrics derived from a confusion matrix are used to quantify this performance.

11.1 Confusion Matrix

The **confusion matrix** is a foundational table that summarizes the performance of a classification model. For a binary classification problem, it has four key components:

- **True Positives (TP):** Correctly predicted positive class.
- **True Negatives (TN):** Correctly predicted negative class.
- **False Positives (FP):** Incorrectly predicted positive class (Type I error).
- **False Negatives (FN):** Incorrectly predicted negative class (Type II error).

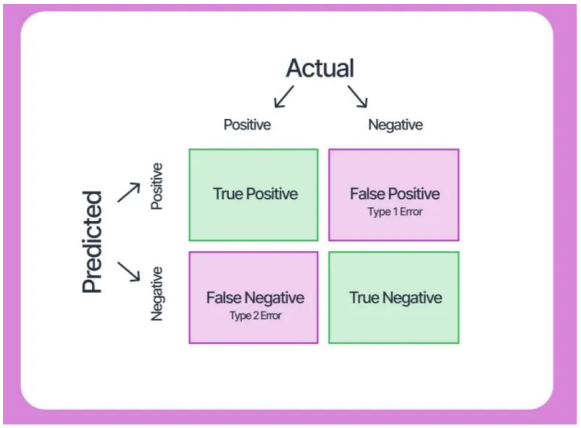


Figure 6: *A typical confusion matrix for binary classification.*

11.2 Core Evaluation Metrics

These metrics provide different perspectives on a model's performance, each with its own strengths and weaknesses.

Accuracy★

Accuracy is the proportion of correct predictions out of the total predictions. While intuitive, it can be misleading for imbalanced datasets.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision and Recall★

Precision and **Recall** are essential for understanding the trade-offs between false positives and false negatives.

- **Precision:** Measures the proportion of positive predictions that were actually correct. High precision indicates a low rate of false positives.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:** Measures the proportion of actual positive cases that were correctly identified. High recall indicates a low rate of false negatives.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-Score★

The **F1-Score** is the harmonic mean of precision and recall. It provides a single metric that balances both, making it particularly useful for imbalanced datasets.

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

11.3 ROC Curve and AUC

The **Receiver Operating Characteristic (ROC) curve** is a plot that visualizes a classifier's performance across all possible classification thresholds.

- The curve plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)**.
- TPR is identical to Recall: $\text{TPR} = \frac{\text{TP}}{\text{TP}+\text{FN}}$
- FPR is the ratio of false positives to all actual negative cases: $\text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}$

The **Area Under the Curve (AUC)** is a single-value summary of the ROC curve. An AUC of 1.0 represents a perfect model, while an AUC of 0.5 indicates a model with no predictive power.

12 Conclusion

[Project Achievements] The FinOptix Summer Project '25 successfully demonstrates a comprehensive framework for portfolio management through:

- Integration of algorithmic trading principles with advanced risk management
- Application of sophisticated financial models (Fama-French, MVO, Black-Litterman)

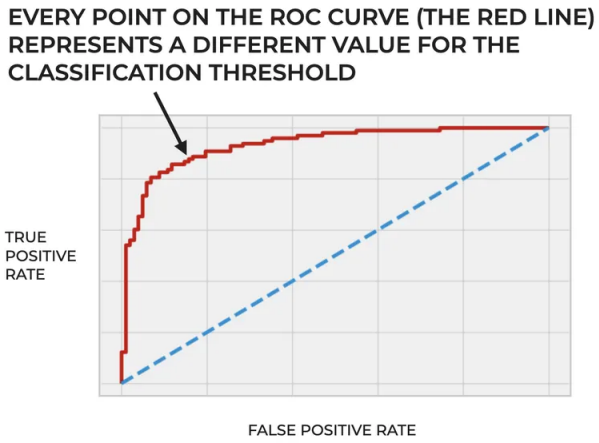


Figure 7: *An example of an ROC curve.*

- Notable improvements in risk-adjusted returns and portfolio stability
- Empirical validation through extensive backtesting across diverse market conditions

13 Results and Conclusion

Till now this report has provided a comprehensive summary of all the concepts learned during this project tenure. This last section focuses on the most important part of the project,i.e the results we obtained on applying the theory to the project.

13.1 Bitcoin Trading Strategy

Assignment 1 consisted of creating and implementing a trading strategy on the BTC data along with strategy optimization in the form of adequate risk management. This was to test our ability to understand market trends and generate and use technical analysis to exploit them.

Key Points

The components of this strategy were:

1. **Custom Volume Indicator:** As volume plays a very important role in the trends of BTC, we created a custom volume indicator to detect drastic changes in volume by taking the difference of a **fast moving exponential average of volume with a slow moving exponential average of volume and comparing it with a threshold.**
2. **OBV:** Using the custom made volume indicator as a denoiser for false signals coupled with OBV for volume trend confirmation and determine the trading direction.
3. **Ichimoku Cloud and EMA Crossover:** We used a combination of Ichimoku cloud and EMA crossover to determine trend direction.
4. **RSI:** Coupling RSI with the above trend indicators to find the strength of the trend, and identify oversold or overbought conditions.
5. **Risk Management Techniques:** In a very volatile environment risk management techniques are very important to protect profits and hedge losses.The various risk management strategies used were:
   - **ATR**
   - **Trailing Stop Loss and Take Profit**
   - **Tradewise Maximum Drawdown Limit**
   - **Intraday Price Change Limit**
   - **Daily Close Price Change Limit**

Results

Static Trade Statistics

Metric	Value
From Timestamp	2020-01-01
To Timestamp	2024-01-01
Total Trades	184
Winning Trades	104
Losing Trades	80
Win Rate (%)	56.52
Long Trades	99
Short Trades	85
Net Profit ($)	5143.79
Gross Profit ($)	5419.79
Average Win ($)	77.38
Largest Win ($)	393.19
Average Loss ($)	-36.30
Largest Loss ($)	-120.19
Max Drawdown (%)	12.71
Avg. Drawdown (%)	1.61
Sharpe Ratio	7.40
Sortino Ratio	18.95
Winning Streak	7
Losing Streak	4
Avg. Holding Time	4 days
Max Holding Time	31 days

Compounding Statistics

Metric	Value
Initial Balance ($)	1000.00
Final Balance ($)	98388.07
Profit Percentage (%)	9738.81
Maximum PNL ($)	11651.37
Minimum PNL ($)	-7140.78
Max Portfolio Balance ($)	105282.21
Min Portfolio Balance ($)	958.79
Max Drawdown (%)	22.42
Average Drawdown (%)	7.06
Time to Recovery (TTR)	75.0 days
Total Fee ($)	10252.86
Leverage Applied	1
Trades Executed	184

13.2 Fama-French and Black-Litterman Model:

In the next part of the project we learn about asset pricing models like Capital Asset Pricing Model(CAPM) and the Fama- French 3 factor and 5 factor model. Along with that we also learned about various portfolio optimization techniques like the Mean Variance Optimization(MVO) and the Black-Litterman Model. Later on we go on to apply these techniques learned by creating a portfolio from a basket of stocks using the Fama- French and Black-Litterman Models.

Key Points

1. **Fama-French 3 Factor Model:** We imported the closing prices of various tickers along with the calculated fama french factors for a given timeframe into a single dataset and then ran the regression model to calculate the betas for all the tickers and plot them. After that we use the long term average of the fama french factors with the calculated betas to get the expected monthly returns.
2. **Black-Litterman Model:** We then use the annualised monthly returns from the Fama French model as prior in the Black-Litterman model. Afterwards based on the investor confidences it generates a posterior which is fed into a **MVO Optimizer** to get a final portfolio distribution.

Code Snippet

```
def calculate_weights(ret_bt, S_bl):
    ef = EfficientFrontier(ret_bl, S_bl)
    ef.add_objective(objective_functions.L2_reg)
    ef.max_sharpe()
    weights = ef.clean_weights()
    return weights
def plot_weights(weights, mcaps):
    mcaps_series = pd.Series(mcaps)
    market_cap_weights = mcaps_series / mcaps_series.sum()

    weights_df = pd.DataFrame({
        'Market Cap Weights': market_cap_weights,
        'Optimal Weights': weights
    })

    weights_df.plot(kind='bar', figsize=(12, 6))
    plt.title('Market Cap Weights vs Optimal Weights')
    plt.ylabel('Weights')
    plt.xlabel('Stocks')
    plt.xticks(rotation=45)
    plt.legend(loc='upper left')
    plt.grid(axis='y')
    plt.tight_layout()
    plt.show()
    def discrete_allocation(weights):
    da = DiscreteAllocation(
        weights,
        prices.iloc[-1],
        total_portfolio_value=initial_capital
    )
    alloc, leftover = da.lp_portfolio()

    for stock in weights:
        if stock not in alloc:
            alloc[stock] = 0

    # print(f"Leftover: ${leftover:.2f}")
    return alloc, leftover
def plot_pie(weights):
    pd.Series(weights).plot.pie(figsize=(10, 10))
```

Code Snippet

```
def calculate_performance(alloc, prices):

    # Step 1: Calculate the expected portfolio return
    daily_returns = prices.copy()
    daily_returns = daily_returns.pct_change()
    daily_returns.dropna(inplace=True)
    mean_daily_returns = daily_returns.mean()
    mean_annual_returns = mean_daily_returns * 252
    expected_annual_returns = mean_annual_returns

    portfolio_return = (
        sum([
            prices.iloc[-1][stock] * alloc[stock] *
    expected_annual_returns[stock]
            for stock in alloc.keys()
        ]) / initial_capital
    ) * 100

    # Step 2: Calculate the portfolio volatility
    cov_matrix = risk_models.sample_cov(daily_returns)

    portfolio_weights = {
        stock: (alloc[stock] * prices.iloc[-1][stock]) / initial_capital
        for stock in alloc.keys()
    }

    weights_array = [
        portfolio_weights[stock]
        for stock in expected_annual_returns.index
        if stock in portfolio_weights
    ]

    portfolio_variance = np.dot(weights_array, np.dot(cov_matrix,
    weights_array))
    portfolio_volatility = np.sqrt(portfolio_variance)

    # Step 3: Calculate the Sharpe ratio
    risk_free_rate = 2
    sharpe_ratio = (portfolio_return - risk_free_rate) /
    portfolio_volatility

    print(f"Expected Portfolio Return: {portfolio_return:.2f}%")
    print(f"Portfolio Volatility: {portfolio_volatility:.2f}")
    print(f"Sharpe Ratio: {sharpe_ratio:.2f}")
```

Code Snippet

```
market_prior = black_litterman.market_implied_prior_returns(mcaps, delta, S)

viewdict = {
    "AAPL": 0.25, "MSFT": 0.15, "AMZN": 0.3, "NVDA": 0.6, "GOOGL": 0.4, "TSLA": 0.5,
    "BRK-B": 0.15, "JNJ": 0.04, "WMT": 0.02, "JPM": 0.035, "V": 0.06, "PG": 0.01, "UNH": 0.015,
    "HD": 0.025, "BAC": 0.05, "MA": 0.04, "DIS": 0.08, "XOM": 0.04, "KO": 0.06, "PFE": 0.015,
    "CVX": 0.02, "MRK": 0.035, "PEP": 0.04, "ABBV": 0.07, "TMO": 0.02, "LLY": 0.025, "AVGO": 0.05,
    "CSCO": 0.04, "COST": 0.03, "DHR": 0.03, "ACN": 0.06, "MCD": 0.04, "NKE": 0.05, "WFC": 0.04,
    "NEE": 0.06, "INTC": 0.03, "TXN": 0.02, "LIN": 0.035, "MDT": 0.025, "HON": 0.03,
    "LOW": 0.01, "PM": 0.015, "IBM": 0.15, "UPS": 0.02, "QCOM": 0.25, "AMGN": 0.0025,
    "ORCL": 0.03, "RTX": 0.005, "NFLX": 0.02, "BA": 0.01
}

# Posterior estimate of returns
ret_bl = bl.bl_returns()
rets_df = pd.DataFrame(
    [market_prior, ret_bl, pd.Series(viewdict)],
    index=["Prior", "Posterior", "Views"]
).T

S_bl = bl.bl_cov()
plotting.plot_covariance(S_bl)
weights1 = calculate_weights(ret_bl, S_bl)
alloc1, left_over1 = discrete_allocation(weights1)
plot_weights(weights1, mcaps)
print("\n")
plot_pie(weights1)
calculate_performance(alloc1, prices)
```

13.2.1 Results:

Portfolio 1:

Expected Portfolio Return:	34.48%
Portfolio Volatility:	1327.47
Sharpe Ratio:	0.02

Portfolio 2:

Expected Portfolio Return:	33.18%
Portfolio Volatility:	1472.44
Sharpe Ratio:	0.02

Portfolio 3:

Expected Portfolio Return:	35.78%
Portfolio Volatility:	1217.92
Sharpe Ratio:	0.03

Portfolio 4:(With Fama French Returns)

Expected Portfolio Return:	46.42%
Portfolio Volatility:	244.49
Sharpe Ratio:	0.18

13.3 Stock Selection Function

13.4 Final Implementation

Acknowledgments

We extend our sincere gratitude to the Finance and Data Analytics Club mentors for their invaluable guidance and support throughout this summer project. Special thanks to the SNT council for giving us this wonderful opportunity to learn and expand our skill set via hands on learning.